

INSTITUTO TÉCNICO SALESIANO VILLADA  
ESPECIALIDAD EN INFORMÁTICA



**Apunte de Git**  
CONTROL DE VERSIONES

**DOCENTES** : Cortesini Perez Luciano Tomas.  
Marchisone Mateo.  
**CÁTEDRA** : Programación I.  
**CURSO** : 4to año C

CÓRDOBA, ARGENTINA  
28 de abril de 2026

# Introducción

## 1.1. Que es git?

GIT (Global Information Tracker) es un sistema de control de versiones distribuido, gratuito y de código abierto, diseñado para gestionar el historial de cambios en el código fuente de proyectos de software de forma rápida y eficiente. Permite a los desarrolladores rastrear modificaciones, volver a versiones anteriores y trabajar colaborativamente sin sobrescribir el trabajo de otros. Git utiliza un modelo basado en instantáneas (snapshots), donde cada confirmación (commit) representa el estado completo del proyecto en un momento determinado.

Cuando decimos que Git es un sistema de control de versiones **distribuido**, significa que cada copia del repositorio contiene el historial completo del proyecto. No dependemos de un único servidor para trabajar: podemos hacer commits, crear ramas y consultar historial sin conexión. Si un servidor remoto falla, cualquier copia del repositorio sirve para recuperar la totalidad del contenido.

## 1.2. Que es GitHub?

**GitHub** es una plataforma en línea para alojar repositorios Git y colaborar con otras personas. Permite compartir código, revisar cambios, trabajar en equipo mediante ramas y pull requests, y mantener un respaldo remoto del proyecto.

Es importante distinguirlos: **Git no es GitHub**. Git es la herramienta que funciona en nuestra computadora (local), mientras que GitHub es un servicio web que usa Git para facilitar la colaboración y la publicación de repositorios. Se puede usar Git sin GitHub, y también existen otras plataformas similares (por ejemplo, GitLab o Bitbucket).

## 1.3. Por que usamos git?

Usar Git y GitHub mejora el flujo de trabajo porque permite:

- trabajar con seguridad, ya que los cambios quedan registrados y se pueden deshacer;
- experimentar sin perder avances, creando commits frecuentes y ramas;
- colaborar de forma ordenada, integrando aportes de varias personas;
- mantener respaldo y trazabilidad del proyecto en el tiempo.

# Conceptos Básicos

## 2.1. Repositorio

Un repositorio en Git es un directorio que contiene tu proyecto junto con toda su historia: cada cambio que se hizo, cuándo, quién lo hizo y por qué. Técnicamente, es una carpeta normal con un subdirectorio oculto llamado `.git` donde Git guarda toda esa información. Ese `.git` es el corazón del repositorio: sin él, sería solo una carpeta común. Hay dos tipos principales:

**Local** — vive en tu máquina. Ahí trabajas, hacés commits, creás ramas. Es completamente funcional sin internet.

**Remoto** — vive en un servidor (GitHub, GitLab, etc.). Sirve como punto central para compartir el trabajo con otros o como respaldo.

## 2.2. Commit

Un commit es una instantánea (foto) del estado de tu proyecto en un momento dado, guardada permanentemente en la historia del repositorio. Cada vez que hacés `git commit`, Git toma todos los cambios que preparaste (los que están en el staging area) y los congela en un paquete inmutable que incluye:

- **El contenido** — cómo quedaron los archivos en ese momento.
- **Metadatos** — autor, fecha y hora.
- **Un mensaje** — descripción de qué cambió y por qué.
- **Una referencia al commit anterior** — así se forma la cadena de historia.

Esa cadena es clave: cada commit apunta al que lo precedió, formando una línea de tiempo que podés recorrer hacia atrás en cualquier momento.

Un commit es como el botón de guardar partida en un videojuego. No solo guardás el progreso actual, sino que podés tener múltiples puntos de guardado y volver exactamente a cualquiera de ellos si algo sale mal.

Git identifica cada commit con un hash SHA-1, un código único como `a3f4c91....`. Ese hash es calculado a partir del contenido del commit, garantizando que el contenido no fue alterado, lo que hace a los commits confiables e inmutables.

El **commit message** debe describir de forma clara qué se cambió y por qué. Buenas prácticas recomendadas:

- usar frases breves y concretas;
- escribir en imperativo (por ejemplo: “Agrega README.md”);
- evitar mensajes genéricos como “cambios” o “arreglos”;
- si hace falta, agregar una descripción más larga explicando contexto.

## 2.3. Áreas de un proyecto Git

El flujo de trabajo de Git se organiza en tres áreas principales: el **working directory** (area de trabajo), donde se realizan las modificaciones; el **staging area** (area de preparación), donde se seleccionan los cambios que formarán parte del próximo commit; y el **local repository** (repositorio local), donde se almacenan de forma permanente las instantáneas confirmadas. Este enfoque permite un control preciso sobre qué cambios se registran en cada commit.

En resumen, el flujo de trabajo típico en Git implica:

- **working directory**: directorio de trabajo donde se crean, editan o eliminan archivos;
- **staging area**: área intermedia donde se seleccionan de forma explícita los cambios que formarán parte del próximo commit;
- **local repository**: repositorio local almacenado en `.git`, donde Git guarda el historial completo del proyecto.

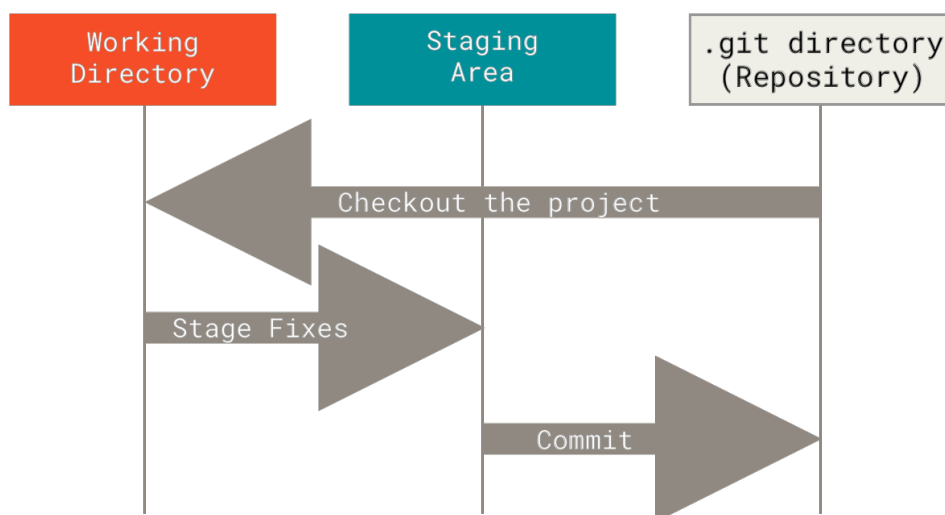


Figura 2.1: Flujo de trabajo en Git

Estas áreas permiten clasificar el estado de los archivos en el proyecto:

- **untracked**: archivo nuevo que Git aún no está siguiendo;
- **modified**: archivo rastreado que fue modificado en el directorio de trabajo, pero todavía no fue agregado al área de preparación;
- **staged**: archivo cuyos cambios fueron agregados al **staging area** y quedarán incluidos en el próximo commit;
- **committed**: cambios almacenados permanentemente en el repositorio local como parte del historial.

Además, al colaborar suele aparecer un **remote repository** (por ejemplo en GitHub), que es la copia alojada en un servidor para compartir cambios con otras personas.

## 2.4. Branch

Una branch (o rama) en Git es una **línea de tiempo paralela** dentro de tu proyecto. Las ramas son utilizadas para desarrollar funcionalidades aisladas unas de otras. La rama main o master es la rama "por defecto" cuando creas un repositorio, y es donde se encuentra la **versión estable del código**. Durante el desarrollo, es común crear ramas para trabajar en nuevas características, corregir errores o experimentar sin afectar la rama principal. Una vez que el trabajo en una rama está completo y probado, se puede integrar de nuevo a main mediante un merge. Este enfoque es útil también a la hora de trabajar colaborativamente, ya que cada desarrollador puede tener su propia rama para sus cambios, evitando modificar el código sobre el que alguien más está trabajando.

Cada rama es esencialmente una referencia o puntero a un commit específico en la historia del proyecto. Cuando haces un commit en esa rama, el puntero se mueve hacia adelante, estando siempre apuntando al último commit de esa línea de desarrollo.

## 2.5. Puntero HEAD

Git utiliza un puntero llamado HEAD para **indicar en qué lugar del árbol estás parado actualmente**. Normalmente HEAD apunta a la rama activa, lo que significa que cualquier commit que hagas se agregará a esa rama. Es posible apuntar HEAD a un commit específico, este estado se llama "detached HEAD" y es útil para revisar versiones anteriores, pero no nos permite hacer commits permanentes sin crear una nueva rama.

Veremos que el comando `git checkout` se usa para cambiar de rama o para moverse a un commit específico, lo que no es más que mover el puntero HEAD.

## 2.6. Merge

**Merge** significa unir dos ramas. Existen dos formas de hacer un merge.

- **Fast-forward** (avance rápido): Si la rama de destino no tiene nuevos commits desde que se creó la rama que queremos unir (las ramas no se han separado), Git simplemente mueve el puntero de la rama de destino hacia adelante, apuntando al mismo commit que la rama que estamos uniendo. En este caso, no se crea un nuevo commit de merge y la historia es mantenida lineal.
- **No-fast-forward** (sin avance rápido): Si ambas ramas avanzaron por separado, Git crea un nuevo commit especial que tiene dos "padres" el último commit de cada rama. Es la unión real de las dos historias.

Por defecto, git intentará realizar un merge fast-forward.

Al realizar un merge, es posible que surja un conflicto: esto se da si las dos ramas modificaron el mismo fragmento del mismo archivo de formas distintas, Git no sabe cuál versión conservar y te pide que lo resuelvas a mano. Ya sea eligiendo una de las dos versiones, o combinándolas.

# Usando git en la práctica

## 3.1. Instalación

## 3.2. Configuración inicial

## 3.3. Como obtener ayuda

Para buscar ayuda rápida de comandos:

- `git help <comando>` (ejemplo: `git help commit`);
- `git <comando>--help`;
- `git <comando>-h` para ver un resumen corto de opciones.

## 3.4. Creando mi primer repositorio

## 3.5. Subir repositorio a github

## 3.6. workflow